

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

доцент, канд. техн. наук
должность, уч. степень, звание

подпись, дата

А. В. Бржезовский
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №7

Триггеры

по курсу: Методы и средства проектирования
информационных систем и технологий

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков
инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Получение навыков создания и использования триггеров в PostgreSQL для реализации активной целостности данных, автоматической обработки операций INSERT/UPDATE/DELETE и формирования производной статистики.

2 Задание и исходные данные

Вариант 5.

Создайте базу данных для хранения следующих сведений: ВУЗ, студент, группа, факультет, конференция, тема доклада, программа конференции.

Ранее была создана база данных представленная на рисунке 1.

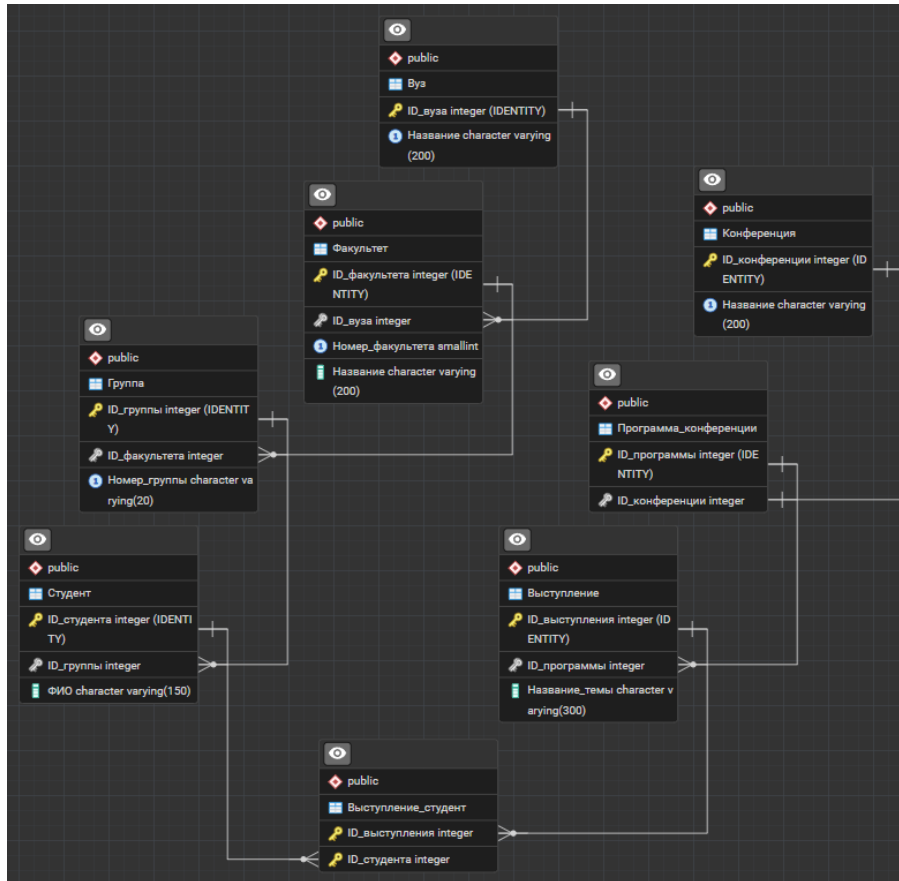


Рисунок 1 – Схема базы данных

На базе данных предметной области «ВУЗ, студент, группа, факультет, конференция, тема доклада, программа конференции» по аналогии с разделом 7.3 методического пособия реализовать: триггеры видов after и instead of для операций insert, update, delete, триггер, выполняющий вычисления и формирование статистики, а также SQL-операторы, демонстрирующие запуск триггеров и результаты их работы.

3 Ход выполнения задания

В скрипте созданы служебные объекты: таблица "Журнал_изменений_студент" для фиксации изменений по студентам и таблица "Статистика_участий_по_конференции" для хранения числа участия студентов в выступлениях по каждой конференции.

Далее реализованы триггеры AFTER на таблицу "Студент". Триггер AFTER INSERT контролирует ограничение численности группы (не более 20 студентов), при превышении лимита генерирует исключение и при корректной вставке добавляет запись в журнал изменений. Триггер AFTER UPDATE при переводе студента в другую группу повторно проверяет ограничение численности и записывает в журнал старые и новые значения ФИО и ID группы. Триггер AFTER DELETE после удаления студента проверяет, остались ли студенты в его группе, удаляет запись из таблицы "Группа", если группа стала пустой, и фиксирует факт удаления в журнале изменений.

Для демонстрации триггеров INSTEAD OF создано представление v_student_editor, объединяющее данные студента, группы, факультета и вуза, и на него назначены триггеры INSTEAD OF INSERT, INSTEAD OF UPDATE и INSTEAD OF DELETE. Триггер INSTEAD OF INSERT по номеру факультета и вузу определяет требуемый факультет, при отсутствии нужной группы создаёт её и вставляет студента в базовую таблицу "Студент". Триггер INSTEAD OF UPDATE обеспечивает изменение ФИО и перевод студента между группами через представление и при необходимости автоматически создаёт целевую группу. Триггер INSTEAD OF DELETE выполняет удаление студента из базовой таблицы "Студент" по ID.

Также реализован триггер вычисления статистики AFTER INSERT OR UPDATE OR DELETE на таблицу "Выступление_студент". Этот триггер поддерживает таблицу "Статистика_участий_по_конференции" в актуальном состоянии при добавлении, изменении и удалении участия. После создания триггера выполнена инициализация статистики по уже существующим данным.

Query	Query History
1	
2	<code>delete from "Студент"</code>
3	<code>where "ФИО" in ('Тест ЛР7 AFTER', 'Тест ЛР7 AFTER (обновлён)');</code>
4	
5	<code>insert into "Студент" ("ID_группы", "ФИО")</code>
6	<code>select г."ID_группы", 'Тест ЛР7 AFTER'</code>
7	<code>from "Группа" г</code>
8	<code>where г."Номер_группы" = '4226'</code>
9	<code>order by г."ID_группы"</code>
10	<code>limit 1;</code>
11	
12	<code>select</code>
13	<code>с."ID_студента",</code>
14	<code>с."ФИО",</code>
15	<code>г."Номер_группы"</code>
16	<code>from "Студент" с</code>
17	<code>join "Группа" г on г."ID_группы" = с."ID_группы"</code>
18	<code>where с."ФИО" = 'Тест ЛР7 AFTER';</code>
Data Output Messages Notifications	
SQL ID_студента integer ФИО character varying (150) Номер_группы character varying (20)	
1	21 Тест ЛР7 AFTER 4226

Рисунок 4 – Результат срабатывания AFTER INSERT на таблице "Студент"

Query	Query History
1	
2	<code>update "Студент"</code>
3	<code>set "ФИО" = 'Тест ЛР7 AFTER (обновлён)'</code>
4	<code>where "ФИО" = 'Тест ЛР7 AFTER';</code>
5	
6	<code>select</code>
7	<code>с."ID_студента",</code>
8	<code>с."ФИО"</code>
9	<code>from "Студент" с</code>
10	<code>where с."ФИО" = 'Тест ЛР7 AFTER (обновлён)';</code>
Data Output Messages Notifications	
SQL ID_студента [PK] integer ФИО character varying (150)	
1	21 Тест ЛР7 AFTER (обновлён)

Рисунок 5 – Результат срабатывания AFTER UPDATE на таблице "Студент"

Query	Query History
1	
2	<code>select</code>
3	<code>'До удаления' as "Этап",</code>
4	<code>с."ID_студента",</code>
5	<code>с."ФИО"</code>
6	<code>from "Студент" с</code>
7	<code>where с."ФИО" = 'Тест ЛР7 AFTER (обновлён)';</code>
8	
9	<code>delete from "Студент"</code>
10	<code>where "ФИО" = 'Тест ЛР7 AFTER (обновлён)';</code>
11	
12	<code>select</code>
13	<code>'После удаления' as "Этап",</code>
14	<code>case</code>
15	<code>when exists (</code>
16	<code>select 1</code>
17	<code>from "Студент" с</code>
18	<code>where с."ФИО" = 'Тест ЛР7 AFTER (обновлён)'</code>
19	<code>) then 'Запись осталась'</code>
20	<code>else 'Запись удалена'</code>
21	<code>end as "Статус";</code>
22	
Data Output Messages Notifications	
SQL Этап text Статус text	
1	После удаления Запись удалена

Рисунок 6 – Результат срабатывания AFTER DELETE с удалением пустой группы

Query

Query History

1

2

3

4

5

6

select *
from "Журнал_изменений_студент"
order by "ID_записи" desc
limit 30;

Data Output

Messages

Notifications

SQL

	ID_записи (PK) integer	Операция character varying (20)	ID_студента integer	Старое_ФИО character varying (150)	Новое_ФИО character varying (150)	Старый_ID_группы integer	Новый_ID_группы integer	Дата_операции timestamp without time zone
1	6	AFTER DELETE	21	Тест ЛР7 AFTER (обновлён)	[null]	1	[null]	2026-04-29 20:36:25.924028
2	5	AFTER UPDATE	21	Тест ЛР7 AFTER	Тест ЛР7 AFTER (обновлён)	1	1	2026-04-29 20:36:02.449125
3	4	AFTER INSERT	21	[null]	Тест ЛР7 AFTER	[null]	1	2026-04-29 20:35:44.285817
4	3	AFTER DELETE	20	Тест ЛР7 Представление INSE...	[null]	7	[null]	2026-04-29 20:34:19.294213
5	2	AFTER UPDATE	20	Тест ЛР7 Представление INSE...	Тест ЛР7 Представление INSE...	12	7	2026-04-29 20:33:55.581373
6	1	AFTER INSERT	20	[null]	Тест ЛР7 Представление INSE...	[null]	12	2026-04-29 20:33:25.932003

Рисунок 7 – Содержимое таблицы "Журнал_изменений_студент"

Query		Query History	
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			
13			
14			
15			
16			
17			
18			
19			
20			
21			
22			
23			
24			
25			
Data Output Messages Notifications			
ID_конференции (PK) integer	Название_конференции character varying (200)	Количество_участий integer	Дата_обновления timestamp without time zone
1	1 Информатика	7	2026-04-29 20:32:56.394471
2	2 Робототехника и ИИ	7	2026-04-29 20:32:56.394471
3	3 Энергетика и экология	3	2026-04-29 20:37:49.825901

Рисунок 8 – Содержимое таблицы

"Статистика_участий_по_конференции" после операций с участиями

4 Вывод

В ходе работы на базе варианта 5 реализованы триггеры after и instead of для операций insert, update и delete, обеспечивающие активную целостность данных при работе со студентами и группами. Дополнительно создан триггер формирования статистики, автоматически поддерживающий агрегированные данные по конференциям. Получен практический опыт разработки и тестирования триггерной логики в PostgreSQL.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

В данном разделе представлен исходный код моей части программы:

-- 0) Служебные объекты

```
drop trigger if exists trg_student_after_insert on "Студент";  
drop trigger if exists trg_student_after_update on "Студент";  
drop trigger if exists trg_student_after_delete on "Студент";  
drop trigger if exists trg_vs_stats_after_ins_upd_del on "Выступление_студент";
```

```
drop function if exists fn_student_after_insert();  
drop function if exists fn_student_after_update();  
drop function if exists fn_student_after_delete();  
drop function if exists fn_vs_stats_after_ins_upd_del();
```

```
drop function if exists fn_v_student_editor_ins();  
drop function if exists fn_v_student_editor_upd();  
drop function if exists fn_v_student_editor_del();
```

```
drop view if exists v_student_editor;  
drop table if exists "Журнал_изменений_студент";  
drop table if exists "Статистика_участий_по_конференции";
```

```
create table "Журнал_изменений_студент" (  
    "ID_записи" integer generated by default as identity primary key,  
    "Операция" varchar(20) not null,  
    "ID_студента" integer,  
    "Старое_ФИО" varchar(150),  
    "Новое_ФИО" varchar(150),
```

```
"Старый_ID_группы" integer,  
"Новый_ID_группы" integer,  
"Дата_операции" timestamp not null default now()  
);
```

```
create table "Статистика_участий_по_конференции" (  
"ID_конференции" integer primary key,  
"Название_конференции" varchar(200) not null,  
"Количество_участий" integer not null default 0,  
"Дата_обновления" timestamp not null default now()  
);
```

```
=====
```

```
-- 1) AFTER-триггеры на таблицу "Студент"
```

```
=====
```

-- AFTER INSERT: контроль максимальной численности группы + запись в журнал.

```
create or replace function fn_student_after_insert()  
returns trigger  
language plpgsql  
as $$  
declare  
v_cnt integer;  
begin  
select count(*)  
into v_cnt  
from "Студент" s  
where s."ID_группы" = new."ID_группы";
```

```
if v_cnt > 20 then
    raise exception 'В группе ID=% превышен лимит 20 студентов (сейчас %)',
new."ID_группы", v_cnt;
end if;
```

```
insert into "Журнал_изменений_студент" (
    "Операция", "ID_студента", "Новое_ФИО", "Новый_ID_группы"
)
values (
    'AFTER INSERT', new."ID_студента", new."ФИО", new."ID_группы"
);
```

```
return new;
end;
$$;
```

```
create trigger trg_student_after_insert
after insert on "Студент"
for each row
execute function fn_student_after_insert();
```

-- AFTER UPDATE: контроль численности при переводе + запись в журнал.

```
create or replace function fn_student_after_update()
```

```
returns trigger
```

```
language plpgsql
```

```
as $$
```

```
declare
```

```
    v_cnt integer;
```

```
begin
```

```
    if new."ID_группы" is distinct from old."ID_группы" then
```

```

select count(*)
into v_cnt
from "Студент" s
where s."ID_группы" = new."ID_группы";

if v_cnt > 20 then
    raise exception 'После перевода в группе ID=% превышен лимит 20
студентов (сейчас %)', new."ID_группы", v_cnt;
end if;
end if;

insert into "Журнал_изменений_студент" (
    "Операция", "ID_студента", "Старое_ФИО", "Новое_ФИО",
"Старый_ID_группы", "Новый_ID_группы"
)
values (
    'AFTER UPDATE',
    new."ID_студента",
    old."ФИО",
    new."ФИО",
    old."ID_группы",
    new."ID_группы"
);

return new;
end;
$$;

create trigger trg_student_after_update
after update on "Студент"

```

```

for each row
execute function fn_student_after_update();

-- AFTER DELETE: удаление пустой группы + запись в журнал.
create or replace function fn_student_after_delete()
returns trigger
language plpgsql
as $$
begin
    if not exists (
        select 1
        from "Студент" s
        where s."ID_группы" = old."ID_группы"
    ) then
        delete from "Группа" g
        where g."ID_группы" = old."ID_группы";
    end if;

    insert into "Журнал_изменений_студент" (
        "Операция", "ID_студента", "Старое_ФИО", "Старый_ID_группы"
    )
    values (
        'AFTER DELETE', old."ID_студента", old."ФИО", old."ID_группы"
    );

    return old;
end;
$$;

create trigger trg_student_after_delete

```

after delete on "Студент"

for each row

execute function fn_student_after_delete();

--

=====

=

-- 2) INSTEAD OF-триггеры на представление

--

=====

=

create or replace view v_student_editor as

select

s."ID_студента",

s."ФИО",

g."Номер_группы",

f."Номер_факультета",

vz."Название" as "Вуз"

from "Студент" s

join "Группа" g on g."ID_группы" = s."ID_группы"

join "Факультет" f on f."ID_факультета" = g."ID_факультета"

join "Вуз" vz on vz."ID_вуза" = f."ID_вуза";

create or replace function fn_v_student_editor_ins()

returns trigger

language plpgsql

as \$\$

declare

v_id_fakulteta integer;

```

v_id_grupy integer;
begin
  select f."ID_факультета"
  into v_id_fakulteta
  from "Факультет" f
  join "Вуз" vz on vz."ID_вуза" = f."ID_вуза"
  where f."Номер_факультета" = new."Номер_факультета"
         and vz."Название" = new."Вуз";

  if v_id_fakulteta is null then
    raise exception 'Не найден факультет №% в вузе %',
new."Номер_факультета", new."Вуз";
  end if;

  select g."ID_группы"
  into v_id_grupy
  from "Группа" g
  where g."ID_факультета" = v_id_fakulteta
         and g."Номер_группы" = new."Номер_группы";

  if v_id_grupy is null then
    insert into "Группа" ("ID_факультета", "Номер_группы")
    values (v_id_fakulteta, new."Номер_группы")
    returning "ID_группы" into v_id_grupy;
  end if;

  insert into "Студент" ("ID_группы", "ФИО")
  values (v_id_grupy, new."ФИО")
  returning "ID_студента" into new."ID_студента";

```



```

return new;
end;
$$;

create trigger trg_v_student_editor_ins
instead of insert on v_student_editor
for each row
execute function fn_v_student_editor_ins();

create or replace function fn_v_student_editor_upd()
returns trigger
language plpgsql
as $$
declare
v_id_fakulteta integer;
v_id_grupy integer;
begin
select f."ID_факультета"
into v_id_fakulteta
from "Факультет" f
join "Вуз" vz on vz."ID_вуза" = f."ID_вуза"
where f."Номер_факультета" = new."Номер_факультета"
and vz."Название" = new."Вуз";

if v_id_fakulteta is null then
raise exception 'Не найден факультет №% в вузе %',
new."Номер_факультета", new."Вуз";
end if;

select g."ID_группы"

```

```

into v_id_grupy
from "Группа" g
where g."ID_факультета" = v_id_fakulteta
and g."Номер_группы" = new."Номер_группы";

if v_id_grupy is null then
insert into "Группа" ("ID_факультета", "Номер_группы")
values (v_id_fakulteta, new."Номер_группы")
returning "ID_группы" into v_id_grupy;
end if;

update "Студент"
set
"ФИО" = new."ФИО",
"ID_группы" = v_id_grupy
where "ID_студента" = old."ID_студента";

return new;
end;
$$;

create trigger trg_v_student_editor_upd
instead of update on v_student_editor
for each row
execute function fn_v_student_editor_upd();

create or replace function fn_v_student_editor_del()
returns trigger
language plpgsql
as $$

```

```

begin
    delete from "Студент"
    where "ID_студента" = old."ID_студента";

    return old;
end;
$$;

```

```

create trigger trg_v_student_editor_del
instead of delete on v_student_editor
for each row
execute function fn_v_student_editor_del();

```

```

--
=====
=
-- 3) AFTER-триггер статистики (insert/update/delete)
--
=====
=

```

```

create or replace function fn_vs_stats_after_ins_upd_del()
returns trigger
language plpgsql
as $$
declare
    v_id_conf_old integer;
    v_id_conf_new integer;
begin
    if tg_op in ('DELETE', 'UPDATE') then

```

```

select pc."ID_конференции"
into v_id_conf_old
from "Выступление" v
join "Программа_конференции" pc on pc."ID_программы" =
v."ID_программы"
where v."ID_выступления" = old."ID_выступления";

if v_id_conf_old is not null then
    insert into "Статистика_участий_по_конференции" (
        "ID_конференции", "Название_конференции", "Количество_участий",
"Дата_обновления"
    )
    select
        k."ID_конференции", k."Название", 0, now()
    from "Конференция" k
    where k."ID_конференции" = v_id_conf_old
    on conflict ("ID_конференции") do nothing;

    update "Статистика_участий_по_конференции"
    set
        "Количество_участий" = greatest(0, "Количество_участий" - 1),
        "Дата_обновления" = now()
    where "ID_конференции" = v_id_conf_old;
end if;
end if;

if tg_op in ('INSERT', 'UPDATE') then
    select pc."ID_конференции"
    into v_id_conf_new
    from "Выступление" v

```

```

join "Программа_конференции" pc on pc."ID_программы" =
v."ID_программы"
where v."ID_выступления" = new."ID_выступления";

if v_id_conf_new is not null then
insert into "Статистика_участий_по_конференции" (
    "ID_конференции", "Название_конференции", "Количество_участий",
"Дата_обновления"
)
select
    k."ID_конференции", k."Название", 0, now()
from "Конференция" k
where k."ID_конференции" = v_id_conf_new
on conflict ("ID_конференции") do nothing;

update "Статистика_участий_по_конференции"
set
    "Количество_участий" = "Количество_участий" + 1,
    "Дата_обновления" = now()
where "ID_конференции" = v_id_conf_new;
end if;
end if;

if tg_op = 'DELETE' then
return old;
end if;

return new;
end;
$$;

```

```

create trigger trg_vs_stats_after_ins_upd_del
after insert or update or delete on "Выступление_студент"
for each row
execute function fn_vs_stats_after_ins_upd_del();

-- Инициализация статистики по текущим данным.
insert into "Статистика_участий_по_конференции" (
    "ID_конференции", "Название_конференции", "Количество_участий",
    "Дата_обновления"
)
select
    k."ID_конференции",
    k."Название",
    count(vs."ID_студента")::integer as "Количество_участий",
    now()
from "Конференция" k
left join "Программа_конференции" pc on pc."ID_конференции" =
k."ID_конференции"
left join "Выступление" v on v."ID_программы" = pc."ID_программы"
left join "Выступление_студент" vs on vs."ID_выступления" =
v."ID_выступления"
group by k."ID_конференции", k."Название"
on conflict ("ID_конференции") do update
set
    "Название_конференции" = excluded."Название_конференции",
    "Количество_участий" = excluded."Количество_участий",
    "Дата_обновления" = now();

```

```

--
=====

=

-- 4) Демонстрационные вызовы (для скриншотов в отчёт)
--
=====

=

-- INSTEAD OF INSERT (через представление)
-- insert into v_student_editor ("ФИО", "Номер_группы", "Номер_факультета",
"Вуз")
-- values ('Тест ЛР7 Представление INSERT', '4999', 4, 'ГУАП');

-- INSTEAD OF UPDATE (через представление)
-- update v_student_editor
-- set "Номер_группы" = '4550'
-- where "ФИО" = 'Тест ЛР7 Представление INSERT';

-- INSTEAD OF DELETE (через представление)
-- delete from v_student_editor
-- where "ФИО" = 'Тест ЛР7 Представление INSERT';

-- AFTER INSERT / UPDATE / DELETE (через базовую таблицу)
-- insert into "Студент" ("ID_группы", "ФИО")
-- select "ID_группы", 'Тест ЛР7 AFTER'
-- from "Группа"
-- where "Номер_группы" = '4226'
-- order by "ID_группы"
-- limit 1;

```

```

-- update "Студент"
-- set "ФИО" = 'Тест ЛР7 AFTER (обновлён)'
-- where "ФИО" = 'Тест ЛР7 AFTER';

-- delete from "Студент"
-- where "ФИО" = 'Тест ЛР7 AFTER (обновлён)';

-- Демонстрация статистического триггера
-- insert into "Выступление_студент" ("ID_выступления", "ID_студента")
-- select v."ID_выступления", s."ID_студента"
-- from "Выступление" v
-- join "Программа_конференции" pc on pc."ID_программы" =
v."ID_программы"
-- join "Конференция" k on k."ID_конференции" = pc."ID_конференции"
-- join "Студент" s on s."ФИО" = 'Алексеев Иван Сергеевич'
-- where k."Название" = 'Энергетика и экология'
-- order by v."ID_выступления"
-- limit 1
-- on conflict do nothing;

select *
from "Журнал_изменений_студент"
order by "ID_записи" desc
limit 20;

select *
from "Статистика_участий_по_конференции"
order by "ID_конференции";

```